

Federated LoRA Fine-Tuning with gRPC

This project implements a structured Federated Learning system using:

- TinyLlama base model
- LoRA adapters (PEFT)
- PyTorch
- gRPC communication
- Server-side federated averaging

The system allows multiple clients to fine-tune a shared base model locally and send only LoRA adapter weights to a central server. The server aggregates the adapters and distributes an updated adapter back to all clients.

Table of Contents

- 1 . Project Overview
- 2 . What is LoRA
- 3 . What is Federated Learning
- 4 . What is gRPC
- 5 . Project Structure
- 6 . Environment Setup
- 7 . Generating gRPC Proto Files
- 8 . Running the Server
- 9 . Running Client Training
- 10 . Sending LoRA to Server
- 11 . Requesting Global Model
- 12 . Testing the Model
- 13 . Expected Outputs

1 . Project Overview

This system demonstrates:

- Local LoRA fine-tuning on each client
- Adapter transmission via gRPC
- Threshold-based aggregation on server
- Global adapter redistribution
- Iterative federated training rounds

The base model is never shared. Only adapter weights are transmitted.

2 . What is LoRA

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method.

Instead of updating all model parameters, LoRA:

- Freezes the base model
- Inserts small trainable matrices
- Updates only those matrices

Advantages:

- Small model updates
- Low memory usage
- Faster training
- Ideal for federated systems

In this project, each client trains only LoRA weights and sends:

- adapter_model.safetensors
 - adapter_config.json
-

3 . What is Federated Learning

Federated Learning is a distributed training approach where:

- Multiple clients train locally
- Raw data never leaves client devices
- Only model updates are sent to server
- Server aggregates updates
- Global model is redistributed

This preserves data privacy and reduces data transfer.

4 . What is gRPC

gRPC is a high-performance remote procedure call (RPC) framework.

It works by:

- 1 . Defining service contracts in a .proto file
- 2 . Generating Python code from proto
- 3 . Server implements service methods
- 4 . Clients call remote methods as if local

In this project:

- SendLoRA() → client uploads adapter
- GetGlobal() → client requests global adapter

Communication occurs over TCP within the same WiFi network or across machines if reachable.

5 . Project Structure

```
federated_project/
├── model/
│   └── TinyLlama-1.1B/
├── proto/
│   └── federated.proto
├── server/
│   ├── grpc_server.py
│   ├── aggregate.py
│   ├── client_uploads/
│   └── global_lora/
├── client_a/
│   ├── train.py
│   ├── send_grpc.py
│   ├── request_global.py
│   ├── data.txt
│   ├── local_lora/
│   └── received_global/
├── client_b/
│   └── same structure as client_a
└── test_lora.py
```

6 . Environment Setup

Create virtual environment:

```
python -m venv venv
venv\Scripts\activate
```

Install dependencies:

```
pip install torch transformers peft safetensors grpcio grpcio-tools
```

If compatibility issues occur, ensure NumPy < 2 .

7 . Generating gRPC Proto Files

From project root:

```
python -m grpc_tools.protoc -I proto --python_out=. --grpc_python_out=.  
proto/federated.proto
```

Copy generated files:

- federated_pb 2 .py
- federated_pb 2 _grpc.py

Into:

- server/
 - client_a/
 - client_b/
-

8 . Running the Server

```
cd server  
python grpc_server.py
```

Expected output:

```
Federated Server running on port 50051...
```

If port binding fails, ensure:

```
server.add_insecure_port("0.0.0.0:50051")
```

9 . Running Client Training

Inside client_a:

```
cd client_a
python train.py
```

Expected output:

```
Loading base model...
No global found. Creating fresh LoRA...
trainable params: ...
[Client A] Step 0 | Loss: ...
...
Client A local LoRA saved.
```

Repeat for client_b.

1 0 . Sending LoRA to Server

After training:

```
python send_grpc.py
```

Expected server output:

```
Received LoRA from client_a
Currently received: ['client_a']
```

After second client:

```
Received LoRA from client_b
Threshold reached. Aggregating...
Aggregation complete. Global model updated.
```

Server saves:

```
server/global_lora/
  adapter_model.safetensors
  adapter_config.json
```

1 1 . Requesting Global Model

Each client runs:

```
python request_global.py
```

Expected output:

```
Global model sent.  
Global model saved in received_global/
```

1 2 . Testing the Model

To test base model only:

```
python base_test.py
```

To test with global LoRA:

```
python test_lora.py
```

Expected output:

```
=== LoRA MODEL OUTPUT ===  
<generated response>
```

1 3 . What is Being Aggregated

The server aggregates only:

- adapter_model.safetensors

Using federated averaging:

```
W_global = (W_client1 + W_client2) / N
```

The adapter_config.json is copied since structure is identical across clients.

The base model is never transmitted or modified on the server.

1 4 . Federated Training Cycle

Round 1 :

- 1 . Client A trains
- 2 . Client B trains
- 3 . Both send adapters
- 4 . Server aggregates
- 5 . Clients request global
- 6 . Clients retrain using global

This loop continues for multiple rounds.

Summary

This project demonstrates:

- Parameter-efficient federated fine-tuning
- gRPC-based distributed communication
- Threshold-triggered aggregation
- Global model redistribution
- Privacy-preserving training workflow

It can be extended with:

- Weighted aggregation
- Model versioning
- Secure TLS channels
- Client authentication
- Differential privacy